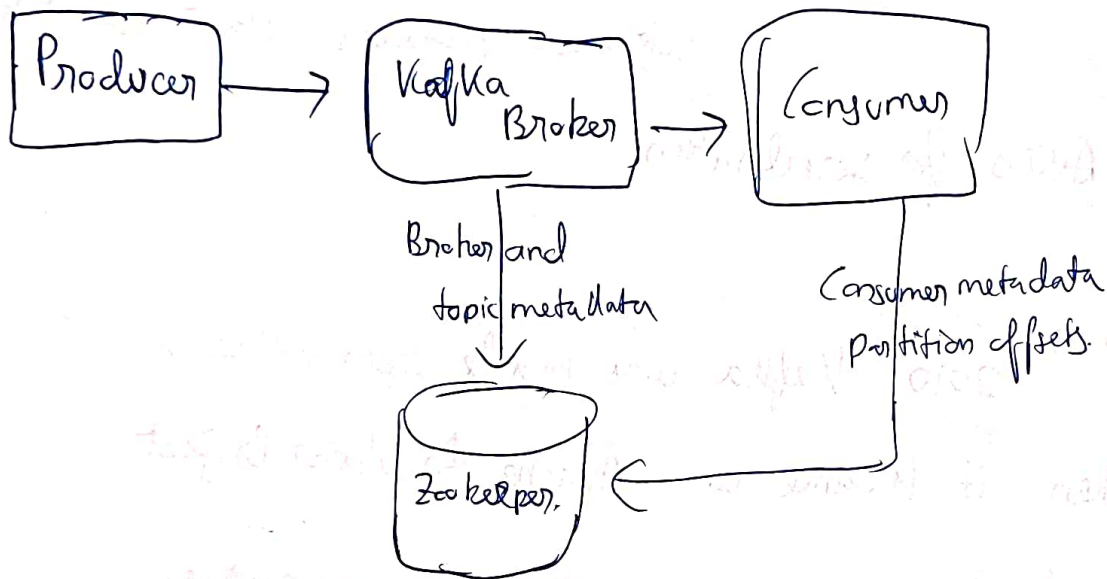


(Chapter 2)

Installing Kafka and some OS Fundamentals.

** Apache Kafka is a Java application.



Zookeeper is to store metadata about Apache Kafka cluster.

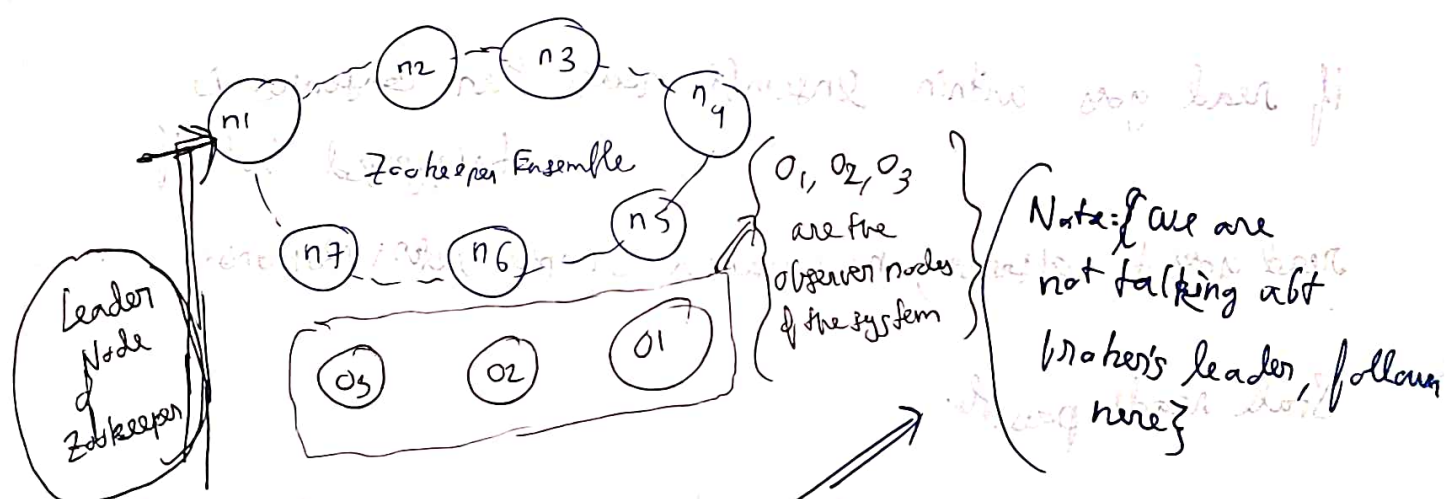
(Zookeeper Ensemble) $\Rightarrow$ Zookeeper cluster called Ensemble.

Due to balancing algorithm, it is recommended for ensemble to have odd number of nodes, as majority of ensemble nodes work as a quorum to respond to requests [Consensus Protocol].

Server $\rightarrow$ 3, 5, - soon

It's recommended not to have more than 7 nodes, else performance goes down due to consensus protocol and quorum agreement.

Zookeeper is queried very frequently for the metadata of the system. So if these 7 (max) nodes of the ensemble are not able to handle load then additional Observer nodes are used to handle read traffic. When traffic goes to observer nodes, then consensus protocol is not triggered, just the simple response.



[Chat gpt] → In Zookeeper there is a leader handling writes, and other ensemble nodes are responsible for handling reads.

Few Configurations → (Unit of time.)

- tickTime → 2000 ms
- initLimit → 20 ⇒ qty ⇒ (20 × 2000) = 40 sec (Amount of time to allow followers to connect to leader)
- syncLimit → 5 ⇒ (Amount of time out of sync followers can be with leader) (5 × 2000) = 10 sec

Then I had a doubt in my mind, Let's suppose a Scenario occurs that some partition got assigned to a new consumer and this change is written to leader, but ~~other~~ read goes to a follower which has lag.

The partition will be consumed from some older consumer. or an error $\rightarrow$ No consumer there.

Or maybe an offset lag.

$\downarrow$
I asked this doubt to ChatGPT

if read goes within ensemble nodes, then consensus is triggered but if read goes to observer, no consensus simple return response.

Stale reads possible.

So Kafka is moving out of Zookeeper and Broker internally now is going to handle this logic.

(KIP-500) Kafka Improvement Proposal - 500, is a major Kafka enhancement which is about removing Zookeeper and replace it with KRaft (Kafka Raft, Internal Raft-based mechanism)

I asked for a brief about Raft.

Raft is a consensus algo used in distributed systems,

It handles

Leader Election $\rightarrow$ Yes there is a leader in Raft

Log Replication $\rightarrow$ Write handled by leader and sends updates to followers, once majority of them agree, entry is committed.

I asked how read & write are handled by Raft?

Writes $\rightarrow$ Leader handles write (Like Dynamo DB)
As described above in Log Replication. Writes goes to Leader, once majority of the followers have the log then entry is committed.

Reads $\rightarrow$
(a) Strongly Consistent Reads $\rightarrow$ Only the leader handles writes
(b) Stale reads from followers.

On a single server, let's say your local machine, you can run,

~~leader~~ Zookeeper on it with port 8080 as leader and other followers as 8081.

General Parameters for Proteins.

broker.id $\rightarrow$ Every broker to have a unique id integer to identify it, like 1, 2, 123 etc.

zookeeper.connect → Config to store zookeeper's location,

hostname $\rightarrow$ ip addresses

part $\rightarrow$ 2081 example

path $\rightarrow$ asked chatgpt about it.

Path parameter in zookeeper. Connect config is an optional chroot path, that allows kafka to store metadata in a specific subdirectory within Zookeeper.

without chroot path, (default root path)

zookeeper.connect = zk1:2181 $\rightarrow$ Metadata stored in $\textcircled{/}$ $\rightarrow$ root directory

with a custom chroot path.

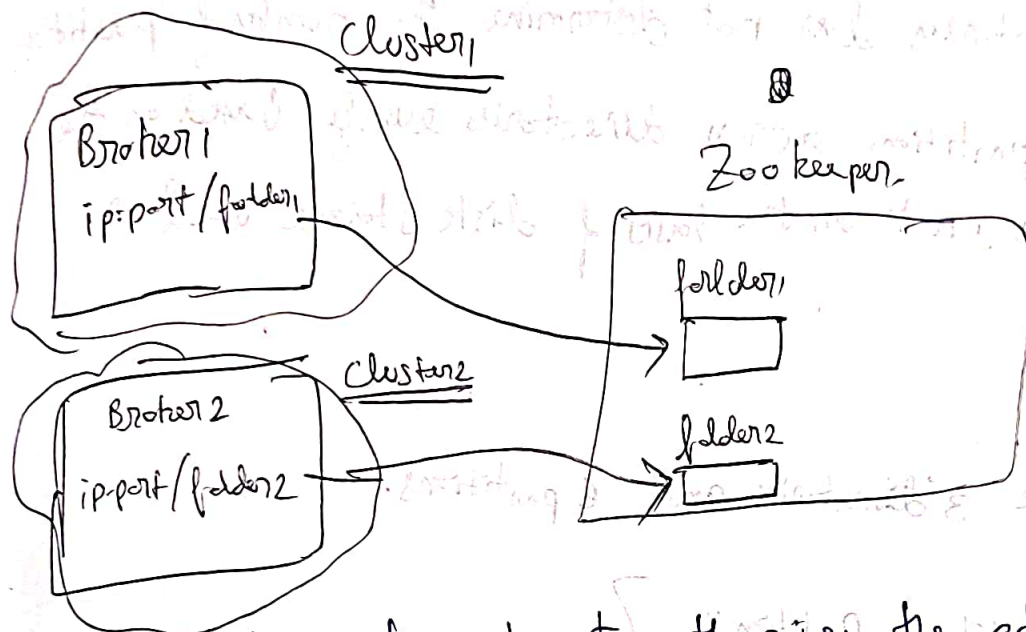
`zookeeper.connect = zk1:2181/kafka-cluster-1`

metadata stored here.

Now in this section, I had a doubt, I was wondering why do ~~we~~ brokers have this parameter?

Well for in Broker the parameter,

zookeeper.connect's path parameter tells where to store the metadata of each broker in the zookeeper.



So specifying this chroot path gives the advantage of and metadata isolation of managing multiple clusters together.

log.dirs → This is the most important parameter of a broker, this will help you understand, how data is stored in a broker. Gear up for it

I read the para and straight up went to chatgpt

Explanation is as follows.

Kafka saves all these messages in the disk. where it stores them.

log.dirs = Locations where you are storing this data.

log.dirs = /data/kafka1, /data/kafka2, /data/kafka3

As you can see we are storing ~~3 partitions~~ 3 directories.

doubt → Does no of log-dirs determine the number of partitions.

No number of directories does not determine the number of partitions. Kafka distributes partitions across directories evenly, based on the number of partitions, not on the basis of disk storage used or remaining.

Let's say you have 3 directories and 6 partitions.

$\left\{ \begin{array}{l} /data/kafka1 \rightarrow \text{Partition 1, Partition 4} \\ /data/kafka2 \rightarrow \text{Partition 2, Partition 5} \\ /data/kafka3 \rightarrow \text{Partition 3, Partition 6} \end{array} \right\}$

If a new partition comes, then it's added to the directory with least number of partitions in it.

Partitions are assigned to log-dirs in the least used fashion.
Assume 3 Partitions.

$/data/kafka1 \rightarrow \text{Partition 1, Partition 4 (2 partitions)}$

$/data/kafka2 \rightarrow \text{Partition 2, Partition 5 (2 partitions)}$

$/data/kafka3 \rightarrow \text{Partition 3, goes here (1 partition)}$

New:
Partition 6

Doubt I asked to chatgpt?

broker has 1 disk, and log-dirs looks something like.

/data/kafka1 → Partition 1, Partition 2

/data/kafka2 → Partition 3, Partition 5

/data/kafka3 → Partition 4.

Now multiple concurrent writes coming to each of the 5 partitions, 1 disk has only 1 disk seeker, how is it making this append only possible and write is so fast

(Explanation)

Let's take the above partition distribution example only, Now messages are being produced.

Producer A → Topic X, Partition 1

Producer B → Topic X, Partition 2

Producer C → Topic Y, Partition 3

Producer D → Topic Z, Partition 4

Producer E → Topic X, Partition 5

Kafka writes these messages to the page cache of OS.

~~How~~ How Page cache looks.

(directory name)

Partition 1 (kafka1) → { msg1, msg2, msg3 }

Partition 2 (kafka2) → { msg1, msg2 }

Partition 3 (kafka2) → { msg --- }

Partition 4 (kafka3) → { msg. --- }

Partition 5 (kafka2) → { msg --- }

this is in-memory and waiting to be flushed to disk.

After OS flushes to disk

/data/kafka1/logs → Partition 1 and 2 flushed from page cache

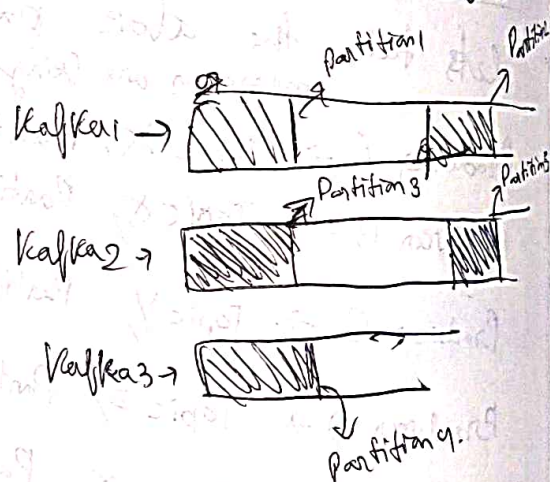
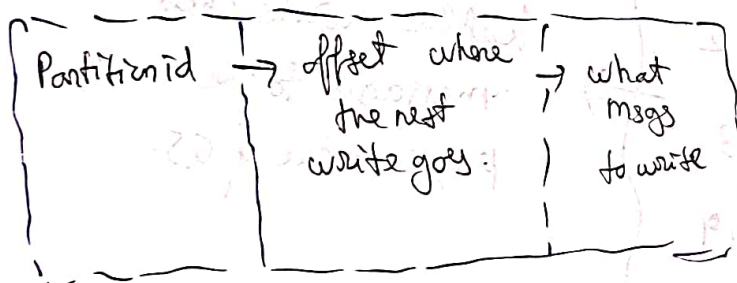
/data/kafka2/logs → Partition 3 and 5 flushed from page cache

/data/kafka3/logs → Partition 4 flushed from page cache

✂ I had the image of page cache, yet I asked chatgpt to explain clearly what is page cache.

OS Page cache is a part of system (RAM) that OS uses to store frequently accessed disk pages. As in the previous pages we ~~have~~ saw

Page cache storing



When consumer comes to read data, first it checks if offset to read is in Page Cache or in disk. If in page cache, read it super fast.

OS holds the data in-memory as writing to disk (I/O call) for so many messages will slow down systems. So then it lets OS thread do this flushing to the disk. (Background process).

{ Kafka itself provides a log flusher thread which manually forcibly flushes this data to disk at some time interval. Optional to use }

`num.recovery.threads.per.data.dir`

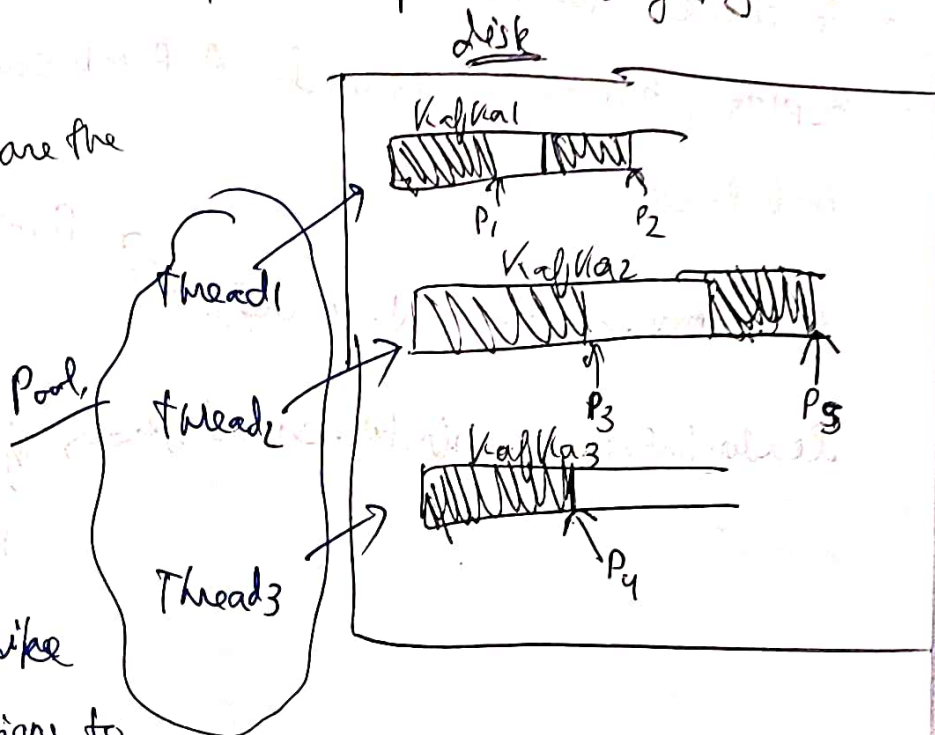
→ Kafka uses a configurable thread pool with a set number of threads

* When starting normally, it would open each partition's log segments.

[Kafka1, Kafka2, Kafka3] are the log directories.

By default 1 thread per directory is there.

It is reasonable to set large number of threads, like equal to number of partitions to parallelize the operations more.



* When shutting down, then clean up log segments properly.

* When starting after failure, then properly check and truncate msgs of partitions which are already consumed.

`auto.create.topics.enable`

3 cases

- 1) producer tries to produce message to new topic
- 2) consumer subscribes to topic that does not exist
- 3) client requests metadata for a topic

if config is true,
Kafka will create a topic with default settings

In production and critical systems, you should set this setting as false

auto.leader.rebalance.enable,

- to ensure that one broker does not get leadership of too many topics, this config is used. A Background thread checks for this imbalance

imbalance.check.interval.seconds

Background thread running at this interval doing validation

leader.imbalance.perbroker.percentage

⇒ If a certain broker leadership percentage goes above this, then, rebalance is triggered.

~~delete~~

Topic Defaults

num.partitions → this config tells what will be the number of partitions created by default when a topic is created

~~***~~

The number of partitions created by a topic can only be increased and cannot be decreased. Kafka does not provide that capability

(How to choose Number of Partitions)

Step 1 → Understand why partition count matters?

Partitions = Parallelism, more partitions → more parallelism. (upside)

Each partition uses memory, CPU. (downside). So you want enough for performance, but not too much that causes overhead.

Step 2 Estimate target throughput.

"How much data this topic will produce/consume per second?"

small use case $\rightarrow 100 \text{ KB/s}$

heavy use case $\rightarrow 1 \text{ GB/s}$

Step 3 Estimate How much a single consumer can handle.

let's say 1 consumer $\rightarrow 50 \text{ MB/sec}$. { Database processing and all that }.

$$1 \text{ GB} / 50 \text{ MB} = \frac{1024}{50} \approx (20 \text{ to } 24 \text{ partitions})$$

Step 4 Consider Producer throughput

let's say 1 producer throughput, we can consider this but it's optional. Producer is very fast and can write high throughput, so we are bottlenecked by the consumer only.

Step 5: Cautions with Keyed Partitions

If $(k, v) \rightarrow \text{message}$, and (k) decides your partitioning logic, then in future if you add new partitions lot of remapping and rebalancing will happen.

Step 6: Think abt broken resources.

more partitions means

* more memory usage per broker { multiple threads writing on the log. dir }

* more file handles

* more time for recovery / leader election

Step 7 Formula

$$\left(\begin{array}{c} \text{total expected throughput} \\ \text{of topic} \end{array} \right) / \left\{ \begin{array}{c} \text{throughput of} \\ \text{1 consumer} \end{array} \right\}$$

Step 8

If you don't have exact numbers, then it's recommended to have 6 GB of data per day.

So (120 GB/day) to the topic

(120/6) $\approx$ 20 partitions.

Also on Kafka the choice of disk, and the IOPS it supports decides the number of writers or partitions we can have we are limited by that as well.

(default-replication-factor)

replication factor $\rightarrow$ how many replicas of the data /
a particular topic should be there
{ partitions }.

Recommendation $\rightarrow (1 + \text{min.insync.replicas})$

(For large clusters we can have) $\rightarrow (2 + \text{min.insync.replicas})$

$\Downarrow$
this is always 1

(log-retention.ms) $\rightarrow$ how long kafka retains the message for
a topic

By default log-retention.hours $\rightarrow$ 168hrs (7 days) (1 week).

two other parameters are there,

(log-retention.minutes, log-retention.ms), if both are
specified the smaller unit will take preference.

(this is always chosen if specified)

(P.T.O)

Then I asked ChatGPT, how are the messages removed after their retention period is over?

The whole process happens like below.

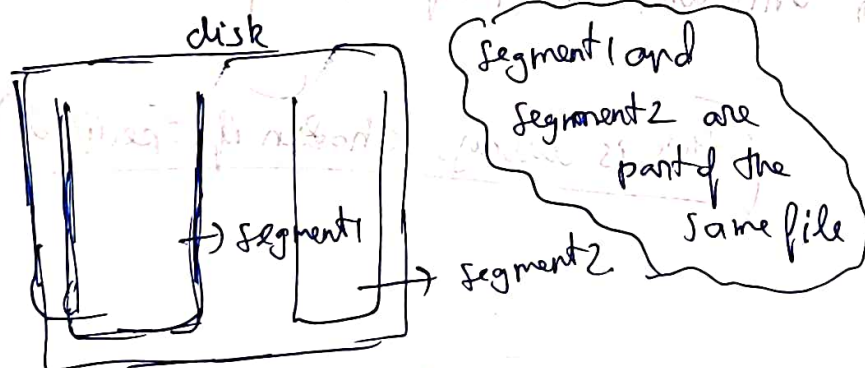
/kafka1 → partition₁, partition₂

/kafka2 → partition₃, partition₄

well each directory has its own partition...

But at disk level, the partition data is being stored in segments.

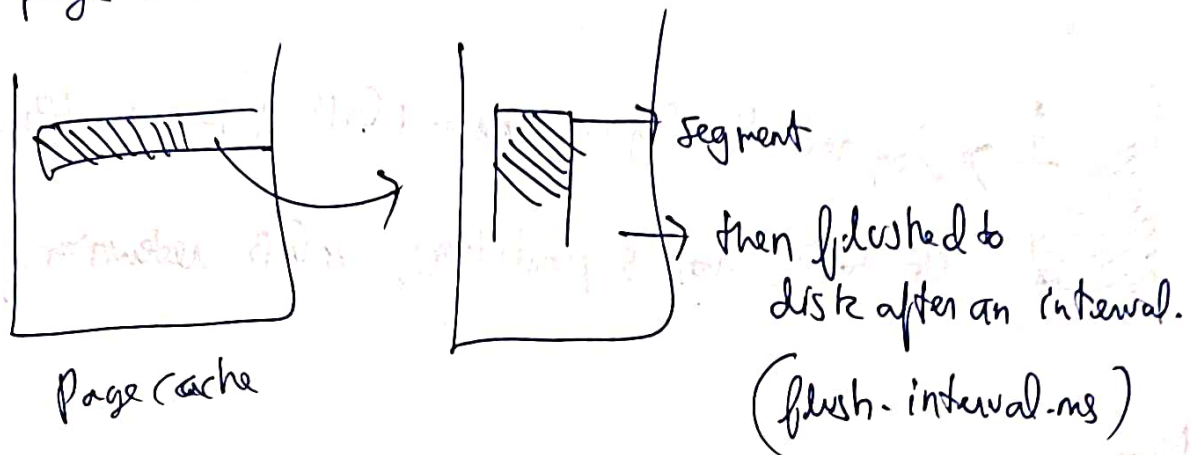
In Operating system we read about segmentation



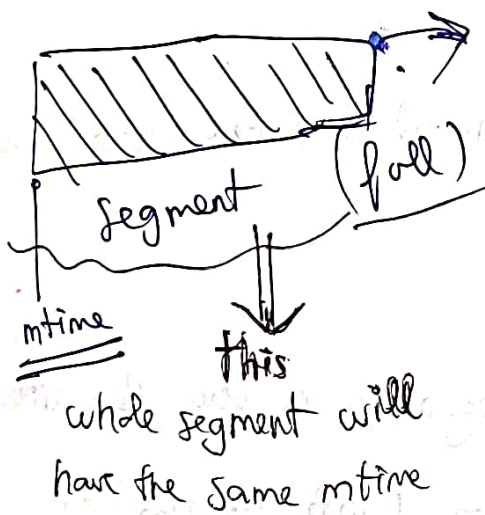
Since contiguous memory is not available so

segments are created and maintained by OS.

So you know group of messages are written to OS page cache.



Well all this messages which are getting written to segment, well the segment itself has a limit size, so it closes after the segment allocated size fills.

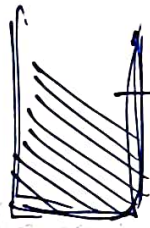


When this segment becomes full, then the segment is closed. and then the whole segment is assigned a last modified time $\rightarrow$ mtime

So when retention period is over, this whole segment is removed. ~~xxx~~ Kafka does not have message level retention, rather it has segment level retention.

Scans all the segments $\rightarrow (currenttime - mtime > retention.ms)$
~~remove the~~ clear the segment.

log-retention-bytes $\rightarrow$ per partition level parameter



$\log\text{-retention-bytes} = 1 \text{ GB (per partition)}$

If topic has 8 partitions, 8 GB retention of topic

Now

$(\log\text{-retention-ms})$ & $(\log\text{-retention-bytes})$

Have the correlated metrics, whichever comes first.

If size is full or messages getting populated are more than $(\log\text{-retention-bytes}) \rightarrow$ clear segment,

if $(\log\text{-retention-ms}) \rightarrow$ if time limit is crossed then ~~not~~ clear segment.

We have to coordinate b/w two.

If retention-ms is too high and msg rate is very less like.

100 KB/sec and retensize is high $\rightarrow$ 1 GB per partition.

then it might happen that the segment is not filling but it became old so it got deleted

that brings us to our next two parameters.

P.T.O

log.segment.bytes →

↑ this log.retention settings previously mentioned works on ~~individual~~ segments and not on individual messages.

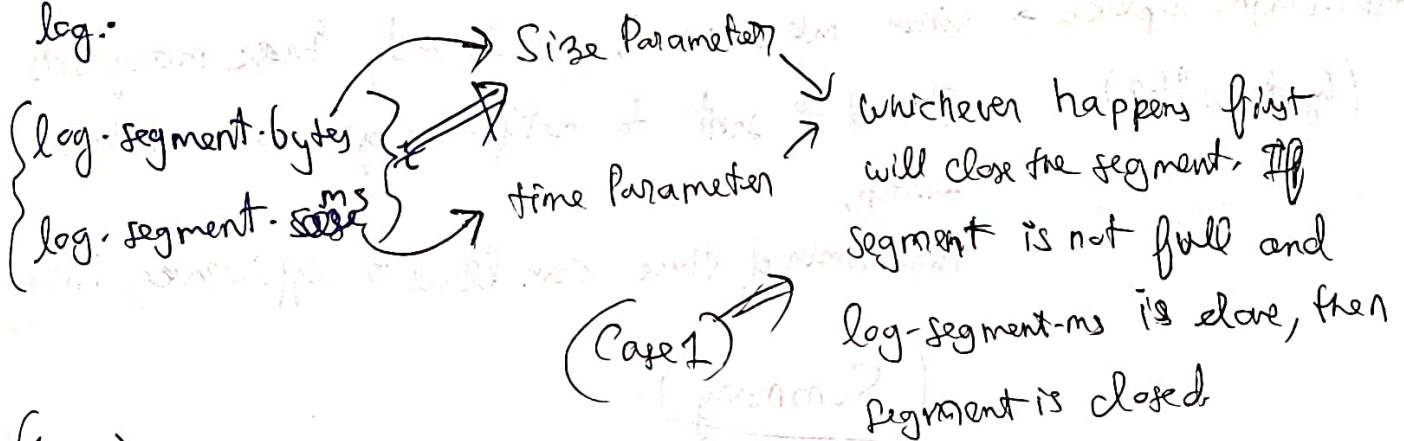
As we saw.

log.retention.ms → clears segments which ~~have~~ have mtime older than retention period.

log.retention.size → per partition level, if size grows beyond this, oldest segments start to get removed.

this parameter, decided when to close a ~~log~~ segment

log:



(Case 2)

⇓ If time is not up but segment size is reaching (log-segments.bytes)

then segment is closed.

Consumer has also a config (fetch.messages.max.bytes) this should be in sync with this

messages.max.bytes → Max size of message a producer can produce to broker. If limit is breached then error is returned. With larger msg size, more disk I/O → more time, ~~performance~~ performance impact

In previous page we saw,

(log.segment.ms) → it is

now (log.roll.ms) → Basically close the segment if after this time ~~the~~ if the segment is not closed.

Disk Impact to

consider →

When multiple segments

~~files~~ are getting closed, because they are not fulfilling the criteria log.segment.bytes, then

can cause performance issues.

min.in.sync.replica → when message is produced, these many acks
(by default 2)
Should be sent to notify msg is successfully written.

High number of these can lead to efficiency issues.

Summary

Topic Configs

num.partitions → # of partitions created
by default when a topic is created

log.retention.ms → min time of a segment is old
clear it

default.replication.factor → Each partition of
topic has a
replica, how many
should they be

log.retention.bytes → per partition level,
1 GB partition, & partition
topic → 8 GB retention,
clear oldest segments

min.insync.replica → min # acks to ~~mark~~
mark message committed

log.segment.bytes → { whichever reaches
first, if size is full or
time is up close the
segment → give min

whichever
1 GB
first &
satisfies
first
remove
it

Criteria for selecting hardware

→ on the broker
64w

Disk throughput → Kafka msgs must be committed to local storage, waiting that atleast one broker has acked it. Faster disk I/O faster throughput lower latency.

SSD → very low latency and lower seek and access time

HDD → cheaper, economical. More storage per unit, ~~fast~~ more storage per disk.

Use HDDs when high storage is there but lower access rate is there, less number of producers. Rate of messages is low.

Use SSDs, when larger number of clients and throughput is more important than storage.

→ Best of both.

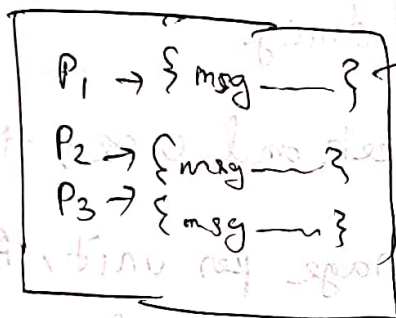
Disk Capacity → Amount of disk required is done by retention period. 1TB traffic each day. 7 days retention.
minimum 7TB disk space required.

(+) 10% for storing other files like metadata etc.

For a topic, have replications on ~~more~~ topics, partitions are stored on different brokers, so if msgs on topic increase and single broker will not have that much load.

(288)

Memory → In the situation, that there is no lag, and Consumer is caught up with the latest messages being produced, then consumer is hitting the in-memory page cache and faster writes are happening



We remember how messages first go to page cache and then go to disk at an interval or space limit

∴ Having more in-memory size on broker improves performance

You know Kafka is written in Java, and it's running and a process is allocated ~~stack~~, a heap memory out of the Ram is allocated to it

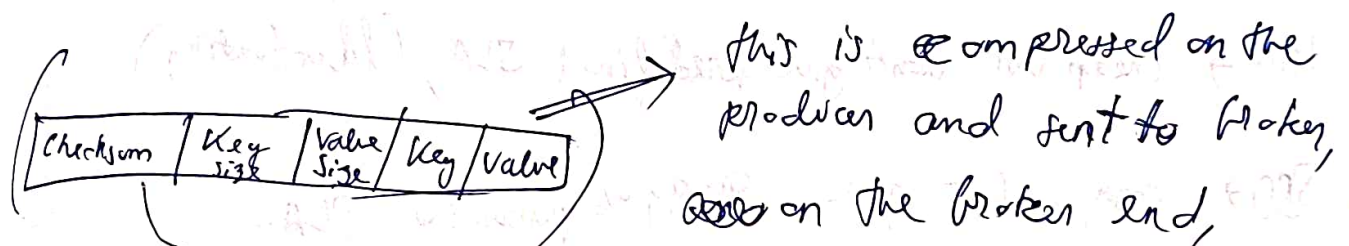


Hence it's not recommended to run another significant app on the broker side by side along with Kafka.

So that Kafka can get a big heap and hence a bigger page cache.

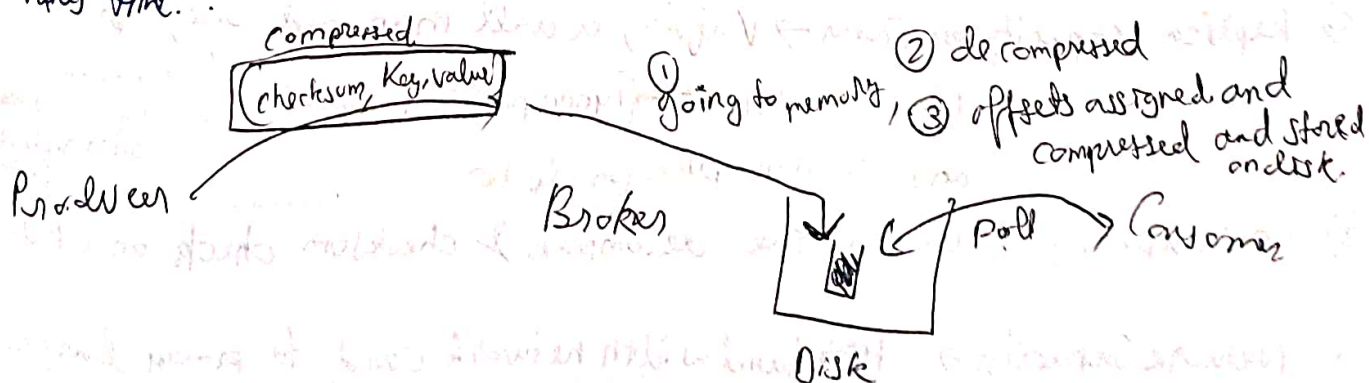
Networking → Network bandwidth, can bottleneck writing from producer to broker's disk and again high replication factor means high network bandwidth is required. So high network card is recommended.

CPU → Do you remember in Apit Bhayani's course we learnt about Data Integrity,



this is decompressed, checksum (Key, value) → recalculated and is matched with checksum.

and then compressed again and stored on disk. This process, is CPU intensive, excluding the disk I/O part. So it takes time.



Kafka in cloud

Cloud providers managing Kafka for you.

You get to decide, number of instances, CPU type, memory and disk you want for it.

Microsoft Azure → They provide VMs and offer machine types to you. they provide variations like HDDs or SSDs.

HDD → cheap but don't give predefined SLA, (fluctuating)

SSD → costly but give 99.99% percentile SLA.

AWS → Costly but provide you with SSDs for highly optimised instances.

(Configuring Kafka Cluster)

- 1) Disk Capacity → what storage each broker machine needs
- 2) Replica Capacity per broker → Kafka, a well managed one, 1 broker can support → (14000 partitions replicas) [that means 14000 partitions with replication factor = 1] and 1 million replica per cluster.
- 3) CPU Capacity → we saw the decompress & checksum check on CPU.
- 4) Network Capacity → High bandwidth network card to process large batches.

Broken Configuration

zoo keeper connect → Having the location and ip address of each broken.

broken.id → Each broken to have unique id.

OS Tuning

Before reading this lets recap a little bit about OS fundamentals, Virtual memory & Swap Space.

Idea → lets say RAM of an instance is 18GB, Yet a process size larger than 18GB is able to run on it. How?

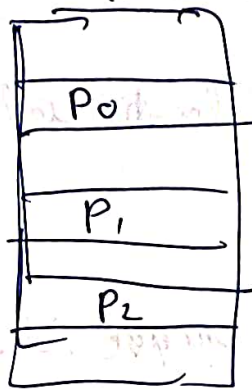
Earlier we used to play games like GTA, IGI which were much bigger than the RAM we had then, How is it possible?

Here comes the concept of virtual memory. Virtual memory is like an inception to the user, making them feel like the RAM is bigger than what they think, by facilitating running processes on the RAM bigger than RAM itself.

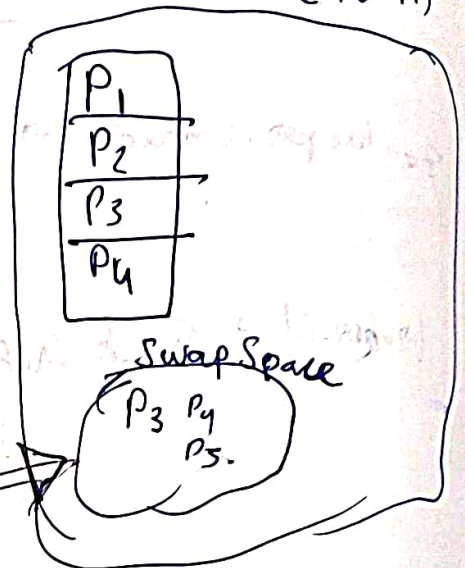
(P40)

How does it do it?

(P_0, P_1, P_2)
are pages
of a process



disk (Rom)



At a given time not all the pages are loaded in RAM,

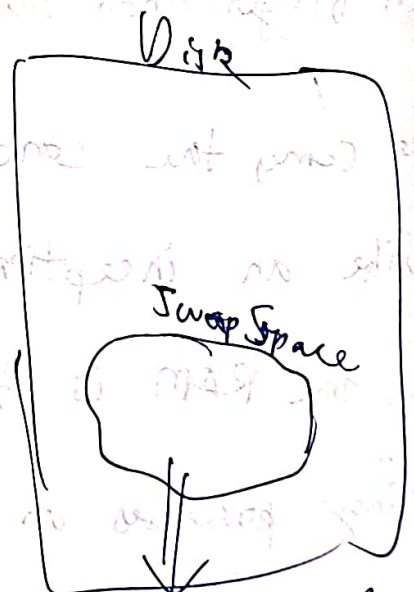
only the required pages of the Process are loaded in RAM,

As an when required ~~the~~ (Pages $\rightarrow P_3/P_4/P_5$) are loaded in memory and Swapped with any of $(P_0/P_1/P_2)$ which is not being used at that instant

This also allows to run two large process of 10GB each maybe to run together.



Few Pages
of Process P_1 10GB
and
few Pages of
Process P_2 of 10GB
are running
together



Few Pages of Process P_1
and few of Process P_2

Now coming back to Kafka

Kafka is a high-throughput low latency system,
so to give that solid high-performance,

Kafka ~~uses~~ ^{avoids} Swappiness → { Note here we are talking about Kafka process JVM, not the page cache where messages are getting written }

fetching pages of the Kafka process from swap space will take a hit on latency. Swapping can kill performance

Config for this.

`vm.swappiness` = 0 → this means never swap Kafka pages.
or process pages

`vm.swappiness` = 1 → tells kernel try not to swap and swap only if it's absolutely necessary.
↓
recommended setting

(`vm.swappiness` = 0) is not recommended because let's say that due to some unforeseen reason Kafka memory usage starts to grow or rather another process, unpredictably started taking memory. If we don't allow swap at all, then it will be out of memory error and server crash. [we compromise by taking a hit and allow swap].

2) Handle Dirty Pages Wisely.

→ Dirty pages are referred to the pages in the page cache, which in-memory messages are stored in RAM. So till the point they are not flushed to disk, are called dirty pages.

we know that Kafka lets OS to handle this flushing of pages.

In case, somehow the OS needs tuning and you feel that tuning can better the performance

vm.dirty-background-ratio = 5%

(RAM 8GB, 5% 400MB)

↳ this means if RAM gets filled by 5% by these pages, then start using background threads to slowly flush the dirty pages to disk.

if we set it 0%, then kernel will constantly and instantly flush it taking a hit on latency

vm.dirty-ratio ≈ (60-80)%

↳ this gives the percentage, that if RAM is filled by (60-80)% due to these pages, then sync call or main thread starts flushing the pages to disk. Blocking call.

Disk

Choice of File System for disk also enables how the impact will be on Kafka.

XFS file system gives better performance in terms of write.

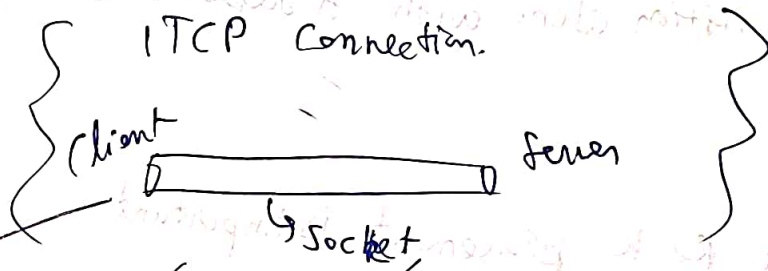
Networking

Kafka handles high-volume, high speed data, so the default Linux settings are not enough, Need to tune it for better output.

What to tune?

1) Socket Buffer Sizes

2) TCP → Specific Buffer Sizes.



{ packets travel to-and-fro from here }

{ if socket size is big
larger number of packets can be transferred. }

Garbage Collector Options

We have read a lot about Garbage Collection ~~Options~~ in Java repository Programming Languages

→ By default Kafka uses Parallel New + CMS { Concurrent Mark and Sweep }

→ But G1 GC gives better performance. / 2 configs are there

Max GC Pause Mills → 200ms

→ GC 1 cycle pauses for this much time, use 20ms for Kafka.

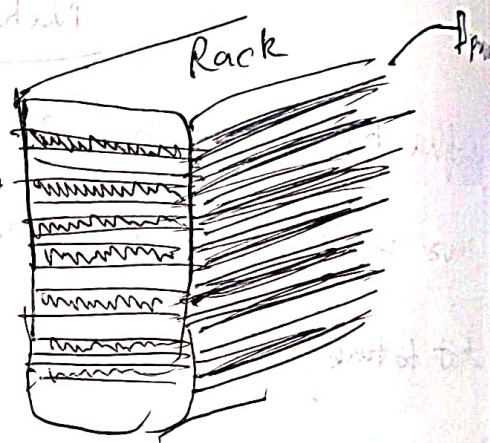
InitialHeapOccupancyPercent → 45% → if 45% of heap is filled triggers GC.

Lastly few things to keep in mind.

Data Center Layout $\rightarrow$ we cannot afford Kafka going down.

By default Kafka is not Rack aware,

So what might happen is that,
all the replicas of all partitions, might
end up on the same rack



(And if power goes down of that Rack,
that partition along with its replica is
gone.

So hence Rack placement is important

place replicas on different racks or different power outputs.

Colocating Applications

on Zookeeper $\rightarrow$ We don't want multiple applications to be

running on the Zookeeper, as it already is a
heap of managing all the brokers.

Kafka only writes metadata abt consumers, producers and topics and
partitions to zookeeper. So having multiple ensemble mode and
consensus protocol might slow down the process. Instead we can have
1 ensemble node with different chroot paths for different
brokers.